

Workshop „Programmieren“

in Python · Arbeitsauftrag

Fachinformatiker*in für Daten- und Prozessanalyse

1. Ausbildungsjahr

Hinweise

Dieses Heft führt euch durch die Workshop-Woche. Die Teile 1 bis 4 gehen wir zum Start gemeinsam durch. Die Teile 5 bis 10 begleiten euch Phase für Phase. Im Anhang findet ihr Bögen, Werkzeuge und Exkurse zum Nachschlagen.

Den Code zum Workshop (Projektvorlage und Beispiele) findet ihr im GitHub-Repository **Workshop**. Wie ihr es holt, steht in Anhang E. Das Heft umfasst **33 Seiten** einschließlich dieses Deckblatts.

Inhalt

WOCHE

1	Worum es geht	2
2	Regeln im Team	3
3	Woran wir ein gutes Ergebnis erkennen	4
4	Checkliste für eure Lösung	5
5	Phase 1: Ideen finden	6
6	Phase 2: Planen	9
7	Phase 3: Umsetzen	11
8	Zwischenreview	13
9	Präsentation	14
10	Reflexion	15

ANHANG

A	Programmieren mit System	17
B	Bogen Zwischenreview	19
C	Bogen Präsentation	20
D	Coding Conventions und Clean Code	21
E	Basics: Arbeiten mit GitHub	24
F	Bibliotheken: Standardmodule und pip	26
G	Fehlerbehandlung mit try und except	28
H	Exkurs: Dateien und Datenströme	30
I	Exkurs: Grafische Oberflächen	32

TEIL 1 · ZIEL

Worum es in dieser Woche geht

Ihr entwickelt im Team ein eigenes Python-Programm: ein kleines Werkzeug für die IT-Administration oder ein Spiel. Ihr findet eine Idee, plant, programmiert, holt euch Feedback und stellt euer Programm am Ende der Woche vor.

AM ENDE DER WOCHE HABT IHR

- ein **lauffähiges Programm**, zu dem jede Person ein eigenes Feature beigetragen hat,
- eine **Präsentation** mit Live-Vorführung, in der jede Person ihren eigenen Code erklärt,
- **Feedback** von den anderen Teams und eine eigene **Reflexion**.

UNTERSCHIEDLICHE ERFAHRUNG IST GEWOLLT

Ihr bringt unterschiedlich viel Erfahrung mit. Das ist kein Problem, sondern Absicht. Jede Person wählt ein Feature, das zu ihr passt: **BASIS** **AUFBAU** **PROFI** . Niemand muss alles können. Wichtig ist, dass du dein Feature **verstehst und erklären kannst**.

Teams

Die Teams stehen fest. Ihr arbeitet in drei Teams mit drei oder vier Personen. Jede Person übernimmt ein eigenes Feature.

TEIL 2 · REGELN

Regeln im Team

Diese Regeln sorgen dafür, dass jede Person etwas lernt und niemand allein alles macht.

1 Jede Person hat ihr eigenes Feature.

In einer eigenen Datei, mit mindestens einer eigenen Funktion. So ist klar, wer wofür zuständig ist.

3 Erst planen, dann programmieren.

Bevor du Code schreibst, steht dein Feature als Teilschritte oder Pseudocode auf Papier (Leitfaden in Teil 6).

5 Steckenbleiben ist normal.

Wenn du nicht weiterkommst, gilt die 10-20-Regel (unten). Es gibt immer einen nächsten Schritt.

7 Feedback ist freundlich und konkret.

Sprecht über Code, nicht über Menschen. „Den Namen `x` verstehe ich nicht“ statt „Das ist schlecht“.

2 Wer tippt, entscheidet.

An deinem Feature hast du die Tastatur. Die anderen helfen als Navigator: Sie stellen Fragen, statt vorzusagen oder selbst zu tippen.

4 Klein anfangen.

Erst Muss, dann Soll, dann Kann. Ein kleines Programm, das läuft, ist mehr wert als ein großes, das nicht läuft.

6 Jede Person erklärt ihren eigenen Code.

Bei der Präsentation zeigst du dein Feature selbst. Schreibe deshalb nur Code, den du erklären kannst.

Die 10-20-Regel: Wenn du feststeckst

0 bis 10 Minuten: allein

Geh die Schritte aus „Programmieren mit System“ durch (Anhang A). Mach das Problem kleiner, teste eine Vermutung mit `print()`.



Nach 10 Minuten: Team fragen

Frag mit der Hilfe-Formel: „Ich möchte X erreichen. Ich habe Y versucht. Ich erwarte A, bekomme aber B.“



Nach 20 Minuten: Ausbilder fragen

Kommt ihr auch zusammen nicht weiter, holt ihr den Ausbilder dazu. Das ist kein Scheitern, sondern effizient.

TEIL 3 · ERGEBNIS

Woran wir ein gutes Ergebnis erkennen

Diese Merkmale beschreiben, wohin ihr arbeitet. Sie sind die Grundlage für das Feedback im Zwischenreview und nach der Präsentation. Größer ist dabei nicht besser: Ein kleines Programm, das sauber läuft und das alle erklären können, ist ein sehr gutes Ergebnis.

1 Es läuft.

Das Programm startet, die Muss-Features funktionieren, und es stürzt bei einer falschen Eingabe nicht ab.

Leitfrage: Was passiert, wenn ich im Menü „x“ eingebe?

2 Man kann es bedienen.

Texte und Menü sind verständlich. Eine Person aus einem anderen Team kommt ohne eure Hilfe zurecht.

Leitfrage: Weiß ich als fremde Person jederzeit, was ich eingeben soll?

3 Die Seminarinhalte sind sichtbar.

Ihr setzt ein, was ihr im Seminar gelernt habt. Die Checkliste in Teil 4 zeigt, was dazugehört.

Leitfrage: Wo im Code steht die for-Schleife, wo die eigene Funktion?

4 Der Code ist sauber.

Namen sind verständlich, Funktionen erledigen jeweils eine Aufgabe, nichts ist unnötig doppelt (Anhang D).

Leitfrage: Könnte jemand aus einem anderen Team ein Feature erweitern?

5 Jede Person hat ihren Teil.

Jede Person hat ein erkennbares Feature beigetragen und kann es zeigen und erklären.

Leitfrage: Was macht diese Funktion, und warum steht diese Zeile hier?

6 Das Vorgehen ist nachvollziehbar.

Teilschritte, Pseudocode und Tests zeigen, wie ihr vorgegangen seid. Feedback aus dem Zwischenreview habt ihr aufgegriffen.

Leitfrage: Was habt ihr nach dem Zwischenreview geändert?

7 Ihr habt als Team gearbeitet.

Abspraken eingehalten, Hilfe gegeben und angenommen.

Leitfrage: Wie habt ihr entschieden, wer was macht?

TEIL 4 · CHECKLISTE

Checkliste für eure Lösung

Das gehört in euer Team-Programm. Nicht jede Person muss alles einbauen, aber im gemeinsamen Programm sollte am Ende alles vorkommen.

PYTHON-GRUNDLAGEN AUS DEM SEMINAR

- ☐ **Variablen** mit verständlichen Namen speichern die Werte, die das Programm braucht.
- ☐ Verschiedene **Datentypen** sind sinnvoll eingesetzt: `int`, `float`, `str`, `bool`.
- ☐ **Eingaben** mit `input()` werden bei Bedarf umgewandelt, z. B. mit `int()` oder `float()`.
- ☐ **Ausgaben** mit `print()` sind verständlich formuliert.
- ☐ **Rechnen oder Texte verarbeiten**, z. B. mit `+`, `*`, `//`, `%` oder `str()`.
- ☐ **Vergleiche** (`==`, `!=`, `<`, `>=` ...) und **Verknüpfungen** (`and`, `or`, `not`).
- ☐ **Verzweigung** mit `if`, `elif` und `else`.
- ☐ **while-Schleife**, z. B. für das Menü oder um eine falsche Eingabe zu wiederholen.
- ☐ **for-Schleife** mit `range()` oder über eine Liste.
- ☐ **Liste**: anlegen, mit `append()` erweitern, mit `len()` zählen, durchlaufen.
- ☐ **Eigene Funktionen** mit Parametern und `return`. Jede Person schreibt mindestens eine.
- ☐ Mindestens ein **Modul** ist eingebunden und genutzt, z. B. `random`, `math` oder `time` (Anhang F).

AUFBAU UND QUALITÄT

- ☐ `main.py` zeigt ein Menü. Jedes Feature liegt in einer **eigenen Datei**.
- ☐ Jede Feature-Datei lässt sich auch **einzelnen starten und testen**.
- ☐ Bei einer **falschen Eingabe** stürzt das Programm nicht ab, sondern fragt erneut.
- ☐ Mindestens eine Funktion ist mit **Schreibtischtest oder Debugger** überprüft.
- ☐ Der **Clean-Code-Check** aus Anhang D ist für jede Datei gemacht.

VORGEHEN

- ☐ Jede Person hat ihr Feature mit **Teilschritten oder Pseudocode** geplant.
- ☐ Mindestens ein Punkt aus dem **Zwischenreview** ist umgesetzt.
- ☐ Die **Präsentation** ist vorbereitet. Jede Person kann ihren Code erklären.

ZUSATZ FÜR FORTGESCHRITTENE (FREIWILLIG)

- ☐ Fehler werden mit **try und except** abgefangen (Anhang G).
- ☐ Daten werden in einer **Datei** gespeichert und wieder geladen (Anhang H).
- ☐ Ein Teil hat eine **grafische Oberfläche** mit `tkinter` (Anhang I).
- ☐ Ein **externes Paket** ist per `pip` installiert und genutzt (Anhang F).

TEIL 5 · PHASE 1

Phase 1: Ideen finden

Ihr entscheidet euch für ein Admin-Werkzeug oder ein Spiel. Auf den nächsten beiden Seiten findet ihr konkrete Vorschläge. Eine eigene Idee ist ausdrücklich willkommen.

1 Allein sammeln

Jede Person notiert für sich drei Ideen. Schaut dafür in den Ideenpool oder überlegt, was euch im Alltag nützen oder Spaß machen würde.

2 Vorstellen

Reihum stellt jede Person ihre Ideen in einem Satz vor.

3 Prüfen

Geht mit den Prüffragen (unten) durch die Ideen. Streicht, was nicht passt. Ideen lassen sich auch kombinieren.

4 Entscheiden

Jede Person hat drei Striche und verteilt sie auf die übrigen Ideen. Bei Gleichstand: kurz diskutieren und noch einmal abstimmen.

5 Festhalten

Haltet Projektname und Idee in einem Satz fest und sammelt erste Features für das Menü.

PRÜFFRAGEN FÜR EINE GUTE IDEE

- Läuft eine einfache Version schon **bis zum Zwischenreview**?
- Gibt es für **jede Person ein eigenes Feature**, auf ihrer Stufe?
- Lässt es sich als **Menü mit mehreren Punkten** bauen?
- Kommen **Seminarinhalte** darin vor (Checkliste, Teil 4)?
- Habt ihr **Lust** darauf?

EIN MENÜ VERBINDET ALLES

Fast jede Idee lässt sich so aufbauen: `main.py` zeigt ein Menü, und jeder Menüpunkt ist ein Feature einer Person.

Beispiel Passwort-Werkstatt:

- 1 – Passwort erzeugen **BASIS**
- 2 – Passwort prüfen **AUFBAU**
- 3 – Merksatz-Passwort **AUFBAU**
- 4 – Passwörter speichern **PROFI**

TEIL 5 · IDEENPOOL

Ideenpool: Admin-Werkzeuge

Kleine Helfer, wie man sie in der IT wirklich braucht. Die Features sind Beispiele, ihr dürft sie ändern und ergänzen.

ADMIN-WERKZEUG

Passwort-Werkstatt

Sichere Passwörter erzeugen und prüfen.

BASIS

Passwort mit fester Länge aus zufälligen Zeichen erzeugen

AUFBAU

Passwort prüfen: lang genug, Ziffer, Großbuchstabe? Mit Begründung

PROFI

Erzeugte Passwörter mit Zweck in einer Datei speichern

ADMIN-WERKZEUG

Geräteverleih

Wer hat welchen Laptop, welches Headset?

BASIS

Alle Geräte einer Liste nummeriert anzeigen

AUFBAU

Gerät ausleihen und zurückgeben, Status „frei“ oder „verliehen“

PROFI

Ausleihe speichern, Rückgabedatum mit `datetime`

ADMIN-WERKZEUG

Benutzerkonten-Generator

Für neue Kolleginnen und Kollegen Konten vorbereiten.

BASIS

Aus Vor- und Nachname einen Benutzernamen bauen, z. B. `max.muster`

AUFBAU

Doppelte Namen erkennen und durchnummerieren, Startpasswort erzeugen

PROFI

Namensliste aus einer Datei lesen, Ergebnis in eine Datei schreiben

ADMIN-WERKZEUG

Helpdesk-Tickets

Störungsmeldungen erfassen und abarbeiten.

BASIS

Ticket mit Titel anlegen, alle Tickets anzeigen

AUFBAU

Priorität und Status („offen“, „erledigt“), nur offene Tickets anzeigen

PROFI

Tickets speichern, Statistik: wie viele offen, wie viele dringend

ADMIN-WERKZEUG

IT-Rechner

Rechnen mit Speichergrößen und Datenraten.

BASIS

Speichergrößen umrechnen: Byte, KB, MB, GB

AUFBAU

Downloadzeit aus Dateigröße und Bandbreite berechnen

PROFI

Zahlen zwischen Dezimal und Binär umrechnen, ohne `bin()`

ADMIN-WERKZEUG

Inventarliste

Verbrauchsmaterial im Blick behalten.

BASIS

Artikel mit Menge anzeigen und eine Menge ändern

AUFBAU

Warnung, wenn ein Bestand unter einen Mindestwert fällt

PROFI

Bestand speichern und laden, Einkaufsliste erzeugen

TEIL 5 · IDEENPOOL

Ideenpool: Spiele

Spiele für die Konsole. Auch hier gilt: Erst eine einfache Version, die läuft, dann ausbauen.

SPIEL

Galgenmännchen

Ein Wort Buchstabe für Buchstabe erraten.

BASIS

Festes Wort, Buchstaben raten, erratene Buchstaben anzeigen

AUFBAU

Zufallswort aus einer Liste, begrenzte Fehlversuche, Galgen als Text

PROFI

Wörter aus einer Datei laden, Schwierigkeitsstufen

SPIEL

IT-Quiz

Fragen rund um Hardware, Netzwerk und Python.

BASIS

Fragen nacheinander stellen, Antwort prüfen, richtig oder falsch melden

AUFBAU

Zufällige Reihenfolge, Auswertung am Ende in Prozent

PROFI

Fragen aus einer Datei, Kategorien wählbar, Highscore

SPIEL

Würfelduell

Zwei Personen würfeln gegeneinander.

BASIS

Beide würfeln einmal, die höhere Zahl gewinnt

AUFBAU

Mehrere Runden, Punktestand, Sonderregel bei einem Pasch

PROFI

Mehrere Würfel mit Kniffel-Kategorien, Highscore-Datei

SPIEL

Schere, Stein, Papier

Gegen den Computer, mit Statistik.

BASIS

Eine Runde gegen den Computer mit `random.choice()`

AUFBAU

„Best of 5“ mit Spielstand und Statistik am Ende

PROFI

Erweiterung um Echse und Spock, Oberfläche mit Knöpfen

SPIEL

Textadventure

Durch Räume laufen, Gegenstände finden, ein Ziel erreichen.

BASIS

Ein Raum mit Beschreibung und zwei Entscheidungen

AUFBAU

Mehrere Räume, Inventar als Liste, Schleife bis zum Ziel

PROFI

Spielstand speichern und später weiterspielen

SPIEL

Mastermind

Einen geheimen Zahlencode knacken.

BASIS

Geheimen dreistelligen Code erzeugen, Tipp einlesen

AUFBAU

Rückmeldung: wie viele Ziffern an der richtigen Stelle

PROFI

Auch „richtige Ziffer, falsche Stelle“ zählen, Versuche begrenzen

EIGENE IDEE?

Gern. Prüft sie mit den Prüffragen aus Phase 1. Wenn ihr für jede Person ein Feature findet und eine einfache Version bis zum Zwischenreview laufen kann, legt los.

TEIL 6 · PHASE 2

Phase 2: Planen

Bevor ihr programmiert, klärt ihr, was euer Programm können soll und wer was baut. Die Fragen sind ein Leitfaden. Wie ihr eure Antworten festhaltet, entscheidet ihr selbst: auf Papier, im Teams-Chat oder als Kommentar im Code.

Fragen fürs Team

1 Was soll euer Programm können?

Sammelt alle Features und ordnet sie ein: **Muss** (läuft bis zum Zwischenreview), **Soll** (kommt danach), **Kann** (wenn Zeit bleibt).

2 Wer baut was?

Jede Person übernimmt ein Muss-Feature. Wählt ehrlich: Ein Basis-Feature, das läuft, bringt mehr als ein Profi-Feature, das nicht fertig wird.

3 Wie passen die Teile zusammen?

Legt für jedes Feature Dateiname und Funktionsname fest, und was die Funktion bekommt und zurückgibt. Dann weiß `main.py`, wie es jedes Feature aufruft.

Fragen für dein Feature

4 Verstehen: Was soll passieren?

Beschreibe dein Feature in einem Satz. Was kommt rein, was soll rauskommen?

5 Konkretisieren: Wie sieht ein Beispiel aus?

Lege eine konkrete Eingabe und das erwartete Ergebnis fest. Denk auch an einen Randfall.

6 Zerlegen und planen: Welche kleinen Schritte gibt es?

Schreibe Teilschritte und den Ablauf als Pseudocode auf. Der erste Teilschritt ist so klein, dass er schnell läuft.

TEIL 6 · PHASE 2

Beispiel: ein Feature durchdacht

So können die Antworten auf die Fragen 4 bis 6 für ein Basis-Feature aussehen. Die Form ist frei, wichtig ist, dass du die Fragen für dich beantwortet hast.

PASSWORT ERZEUGEN BASIS

In einem Satz: Erzeugt ein zufälliges Passwort in der gewünschten Länge.

Eingabe: Länge als Zahl, z. B. 8

Ausgabe: Passwort als Text, z. B. k3T9xQ2a

Randfall: Länge 2 ist zu kurz, also erneut fragen.

Zusammenspiel: Dateiname `passwort.py`, Funktion `erzeuge_passwort(laenge)`, gibt einen `str` zurück.

TEILSCHRITTE

- ☐ 1. Ein einzelnes zufälliges Zeichen ausgeben (`random.choice`)
- ☐ 2. Mit einer `for`-Schleife 8 Zeichen aneinanderhängen
- ☐ 3. In eine Funktion mit Parameter `laenge` packen, Ergebnis mit `return`
- ☐ 4. Länge mit `input()` abfragen und umwandeln
- ☐ 5. Zu kurze Länge abfangen und erneut fragen
- ☐ 6. In `main.py` ins Menü einbauen

Pseudocode für Teilschritt 2 und 3

```
# Pseudocode für Teilschritt 2 und 3
Funktion erzeuge_passwort mit laenge:
    passwort ist zuerst leer
    wiederhole laenge-mal:
        hänge ein zufälliges Zeichen an passwort an
    gib passwort zurück
```

TEIL 7 · PHASE 3

Phase 3: Umsetzen

Jetzt wird programmiert, Teilschritt für Teilschritt. Bis zum Zwischenreview laufen die Muss-Features, bis zur Präsentation ist das Programm fertig.

Projektordner anlegen

Holt euch das Repository **Workshop** (Anhang E). Kopiert den Ordner `projektvorlage` als Projektordner für euer Team, zum Beispiel nach `Dokumente/workshop-team`. So bleibt die Vorlage unverändert.

```
workshop-team/
├── main.py           ← Menü, ruft die Features auf
├── password.py       ← Feature von Person 1
├── pruefer.py        ← Feature von Person 2
├── merksatz.py       ← Feature von Person 3
└── speicher.py       ← Feature von Person 4
```

Jede Person kopiert

`feature_vorlage.py` und benennt die Kopie passend. Eine Person aus dem Team baut die Features in `main.py` ein.

Dateinamen: klein, ohne Umlaute und Leerzeichen, z. B. `password.py`. Sie müssen genau zu dem passen, was `main.py` importiert.

So hängen die Dateien zusammen

`password.py`

```
import random

ZEICHEN = "abcdefghijklmnopqrstuvwxyz23456789"

def erzeuge_passwort(laenge):
    password = ""
    for i in range(laenge):
        password = password + random.choice(ZEICHEN)
    return password

def starte():
    laenge = int(input("Länge: "))
    print("Dein Passwort:", erzeuge_passwort(laenge))

if __name__ == "__main__":
    starte()
```

`main.py` (Ausschnitt)

```
import password

...

if auswahl == "1":
    password.starte()
```

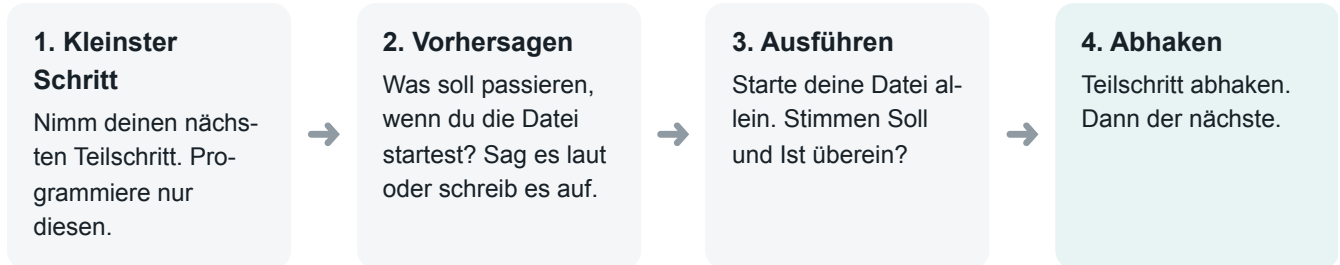
`import password` lädt die Datei `password.py`, die im selben Ordner liegt. Danach ruft `password.starte()` die Funktion aus dieser Datei auf.

`if __name__ == "__main__":` Dieser Block läuft nur, wenn du genau diese Datei startest. So kannst du dein Feature allein testen, ohne dass das Menü der anderen schon fertig sein muss.

TEIL 7 · PHASE 3

Arbeiten in kleinen Schritten

Wer in kleinen Schritten programmiert und nach jedem Schritt testet, findet Fehler sofort und kommt schneller ans Ziel.



Passt etwas nicht, hilft Schritt 7 „Fehler eingrenzen“ aus Anhang A: Stelle eine Vermutung auf und prüfe sie gezielt mit `print()` oder dem Debugger.

Vor dem Zwischenreview: Muss-Features fertig

Bis zum Zwischenreview soll euer Programm starten und die Muss-Features zeigen. Was noch nicht fertig ist, nehmt ihr vorübergehend aus dem Menü. Ein Menüpunkt, der abstürzt, hilft niemandem.

Nach dem Zwischenreview: verbessern, ausbauen, aufräumen

VERBESSERN

Feedback aus dem Zwischenreview einarbeiten. Danach Soll-Features.

AUSBAUEN

Wenn eure Muss-Features laufen, könnt ihr einen Exkurs einbauen (Anhang F bis I).

AUFRÄUMEN

Clean-Code-Durchgang mit dem Check aus Anhang D, danach die Präsentation vorbereiten.

EXKURSE FÜR FORTGESCHRITTENE

Pakete mit `pip` (Anhang F), Fehlerbehandlung mit `try` und `except` (G), Dateien (H) und grafische Oberflächen (I) gehen über das Seminar hinaus. Sie sind **freiwillig** und nur dann sinnvoll, wenn dein Muss-Feature schon läuft. Jeder Exkurs hat eine Einführung zum Lesen und lauffähige Beispiele im Ordner `beispiele` des Repositorys. Wer einen Exkurs einbaut, erklärt ihn bei der Präsentation kurz den anderen.

Dateien im Team austauschen

Ihr arbeitet jeweils an eurer eigenen Datei. So entstehen keine Konflikte.

- Legt im Teams-Kanal eures Teams einen Ordner für den Code an. Jede Person lädt **nur ihre eigene Datei** hoch und ersetzt dort die alte Version.
- Regelmäßig lädt eine Person alle aktuellen Dateien herunter, legt sie in den gemeinsamen Projektordner und startet `main.py`: Läuft alles zusammen?
- Ändert nie die Datei einer anderen Person. Gebt stattdessen einen Hinweis.

TEIL 8 · ZWISCHENREVIEW

Zwischenreview

Je zwei Teams schauen sich gegenseitig Programm und Code an. Ihr bekommt frische Augen auf euer Projekt, und ihr seht, wie andere ein Problem gelöst haben.

Drei Runden

RUNDE	WER REVIEWT WEN?	DAS DRITTE TEAM
Runde 1	Team 1 und Team 2 reviewen sich gegenseitig.	Team 3 arbeitet weiter, der Ausbilder schaut vorbei.
Runde 2	Team 2 und Team 3 reviewen sich gegenseitig.	Team 1 arbeitet weiter.
Runde 3	Team 3 und Team 1 reviewen sich gegenseitig.	Team 2 arbeitet weiter.

So reviewt jedes Team zweimal und bekommt von beiden anderen Teams Feedback.

Ablauf in eine Richtung

1 Ausprobieren

Das Gast-Team bedient das Programm, **ohne Erklärung**. Es denkt dabei laut: „Ich will jetzt ... Ich erwarte ...“. Das Gastgeber-Team schaut zu und schweigt.

2 Code lesen

Das Gastgeber-Team zeigt eine Feature-Datei. Das Gast-Team liest sie mit dem Bogen aus Anhang B und stellt Verständnisfragen.

3 Feedback

Das Gast-Team gibt Feedback nach dem Muster **zwei Sterne und ein Wunsch**: zwei Dinge, die gut gelungen sind, und ein konkreter Verbesserungsvorschlag.

Danach wird getauscht: Das Gastgeber-Team wird zum Gast-Team.

FEEDBACK GEBEN

- Konkret: „Bei Menüpunkt 2 wusste ich nicht, ob ich eine Zahl oder ein Wort eingeben soll.“
- Über die Sache, nicht über die Person.
- Auch Gelungenes benennen, und zwar genau.

FEEDBACK ANNEHMEN

- Zuhören und mitschreiben, nicht rechtfertigen.
- Nachfragen, wenn etwas unklar ist.
- Danach im Team entscheiden, was ihr umsetzt.

Nach dem Review überlegt ihr im Team: Was nehmen wir uns als Nächstes vor? Tragt mindestens einen Punkt unten auf dem Review-Bogen ein und setzt ihn bis zur Präsentation um.

TEIL 9 · PRÄSENTATION

Präsentation

Ihr zeigt, was ihr gebaut habt und wie ihr dabei vorgegangen seid.

Ablauf pro Team

TEIL	INHALT
1. Idee und Plan	Was macht euer Programm, für wen? Wie habt ihr die Features verteilt?
2. Fremde Vorführung	Eine Person aus einem anderen Team bedient euer Programm. Ihr dürft sagen, was sie ausprobieren soll, aber nicht die Tastatur übernehmen.
3. Code-Einblick	Jede Person zeigt eine Funktion aus ihrem Feature und erklärt sie Zeile für Zeile.
4. Rückblick	Was lief gut? Wo seid ihr steckengeblieben, und wie seid ihr weitergekommen?
5. Fragen und Feedback	Die Zuhörenden stellen Fragen und geben Feedback. Die Leitfragen im Bogen Präsentation (Anhang C) helfen dabei.

VORBEREITUNG

- ☐ Programm läuft auf dem Präsentationsrechner
- ☐ Reihenfolge der Sprechenden steht fest
- ☐ Jede Person hat ihre Funktion ausgewählt
- ☐ Eine Vorführung einmal durchgespielt
- ☐ Plan B, falls etwas abstürzt: Screenshots

WARUM EINE FREMDE PERSON VORFÜHRT

Ob euer Programm verständlich ist, merkt ihr erst, wenn jemand es bedient, der es nicht kennt. Jedes Zögern zeigt eine Stelle, an der ein Text oder ein Menü klarer werden kann.

TEIL 10 · REFLEXION

Reflexion

Zum Abschluss denkt jede Person zuerst allein über die Woche nach. Danach sagt jede Person in der Abschlussrunde einen Satz: „Das nehme ich mit ...“

WAS HABE ICH DIESE WOCHE GELERNT, DAS ICH VORHER NICHT KONNTE?

WO BIN ICH STECKENGEBLIEBEN, UND WAS HAT MIR GEHOLFEN WEITERZUKOMMEN?

WELCHER SCHRITT AUS „PROGRAMMIEREN MIT SYSTEM“ HAT MIR AM MEISTEN GEHOLFEN, UND WARUM?

WAS HAT IN UNSEREM TEAM GUT FUNKTIONIERT, WAS WÜRDTE ICH ÄNDERN?

WAS MACHE ICH BEIM NÄCHSTEN PROJEKT ANDERS?

ANHANG

Anhang: Übersicht

Bögen, Werkzeuge und Exkurse zum Nachschlagen.

	ANHANG	WANN BRAUCHST DU IHN?	SEITE
A	Programmieren mit System	Immer, wenn du ein Problem angehst oder feststeckst.	17
B	Bogen Zwischenreview	Beim Zwischenreview, ein Bogen pro Review.	19
C	Bogen Präsentation	Bei der Präsentation, ein Bogen pro präsentierendem Team.	20
D	Coding Conventions und Clean Code	Beim Programmieren und beim Clean-Code-Durchgang.	21
E	Basics: Arbeiten mit GitHub	Wenn ihr das Repository holt, und als Spickzettel für Git-Befehle.	24
F	Bibliotheken: Standardmodule und pip	Sobald du ein Modul wie <code>random</code> brauchst. Der pip-Teil ist ein Exkurs.	26
G	Fehlerbehandlung mit try und except	Wenn falsche Eingaben dein Programm nicht mehr abstürzen lassen sollen.	28
H	Exkurs: Dateien und Datenströme	Für Fortgeschrittene, wenn Daten erhalten bleiben sollen.	30
I	Exkurs: Grafische Oberflächen	Für Fortgeschrittene, wenn ein Feature ein Fenster bekommen soll.	32

EINZELN DRUCKEN

Die Checkliste, die beiden Bögen und die Reflexion liegen im Repository zusätzlich als einzelne PDFs im Ordner `vorlagen`.

Programmieren mit System

Wenn du feststeckst, hilft ein klarer Ablauf. Jeder Schritt liefert etwas, mit dem du weiterarbeiten kannst.

1

VERSTEHEN

Was soll passieren?

Formuliere die Aufgabe in eigenen Worten. Kläre Eingabe, gewünschte Ausgabe und offene Fragen.

Notenliste gegeben. Gesucht: der Durchschnitt.

2

KONKRETISIEREN

Wie sieht ein Beispiel aus?

Lege konkrete Eingaben und erwartete Ergebnisse fest, bevor du Code schreibst.

[2, 3, 1, 4]: Summe 10, Anzahl 4, Ergebnis 2.5.

3

ZERLEGEN

Welche kleinen Teile gibt es?

Teile das Ziel in überschaubare Teilprobleme. Benenne, welcher Teil noch unklar ist.

Liste prüfen, addieren, zählen, teilen, ausgeben.

4

PLANEN

Wie löse ich es auf Papier?

Rechne von Hand. Schreibe den Ablauf in Alltagssprache oder Pseudocode auf.

Summe bei 0 starten. Jede Note dazuzählen.

5

IMPLEMENTIEREN

Was ist der kleinste nächste Schritt?

Programmiere einen Teil, den du bereits erklären kannst. Erweitere ihn anschließend.

Erst eine Note addieren, dann alle in einer Schleife.

6

TESTEN

Stimmen Soll und Ist überein?

Sage das Ergebnis voraus, führe den Code aus und prüfe auch einen Randfall.

[3] ergibt 3.0. [] soll „Keine Noten“ ergeben.

7

FEHLER EINGRENZEN

Welche Vermutung kann ich prüfen?

Teste eine Hypothese gezielt, etwa mit print() oder dem Debugger. Ändere eine Sache auf einmal.

Stimmt die Anzahl? Prüfe zuerst len(noten).

8

REFLEKTIEREN

Welcher Gedanke hat geholfen?

Erkläre deinen Lösungsweg. Halte fest, was du beim nächsten Problem wieder anwenden kannst.

„Die Zwischensummen haben den Fehler gezeigt.“

Du darfst zurückgehen: Ein Test kann zeigen, dass du die Aufgabe neu verstehen oder deinen Plan ändern musst.

SO BITTEST DU KONKRET UM HILFE

„Ich möchte X erreichen. Ich habe bisher Y versucht. Ich erwarte A, bekomme aber B. Ich vermute, dass es an C liegt. Bei D komme ich nicht weiter.“

Dein Werkzeugkasten

Wähle die Methode, die zu deiner aktuellen Frage passt. Dein Ziel ist ein kleiner Schritt, der dir neue Klarheit gibt.

Pseudocode

Vor dem Programmieren

Beschreibe die Logik in Alltagssprache. So klärst du den Ablauf, bevor du Python-Syntax brauchst.

„Für jede Note: zur bisherigen Summe addieren.“

Rubber Duck Debugging

Wenn du den Fehler nicht siehst

Erkläre deinen Code einer Person oder einem Gegenstand laut, Zeile für Zeile. Was erwartest du jeweils?

„Hier setze ich die Summe wieder auf 0. Warum?“

Minimal Reproducible Example

Wenn das Problem unübersichtlich ist

Verkleinere Daten und Code, bis nur ein kleines Beispiel übrig bleibt, das denselben Fehler zeigt.

Statt einer ganzen Anwendung: zwei Zahlen und eine Schleife.

Hypothesengetriebenes Debugging

Bei unerwartetem Verhalten

Formuliere eine Vermutung. Plane einen gezielten Test und vergleiche die Beobachtung mit deiner Vorhersage.

„Die Anzahl ist falsch.“ Test: `print(len(noten))`.

Testgetriebenes Denken

Vor und während der Umsetzung

Lege vorher fest, woran du eine korrekte Lösung erkennst. Prüfe Normalfälle und Grenzen.

[1, 6] soll 3.5 ergeben. Für [] klären wir die Ausgabe.

DREI REGELN FÜR JEDES PROGRAMMIERPROBLEM

01 Mach das Problem kleiner.

Suche einen Teil, den du schon lösen oder gezielt ausprobieren kannst.

02 Vermute nicht nur. Experimentiere.

Prüfe deine Hypothese mit möglichst wenig Code und einer klaren Vorhersage.

03 Stelle konkrete Fragen.

Beschreibe, was du erreichen willst, was du beobachtest und welche Stelle unklar ist.

Du musst die Lösung noch nicht kennen, um den nächsten Schritt zu finden.

ANHANG B · VORLAGE

Bogen Zwischenreview

Füllt ihn als Gast-Team aus und gebt ihn danach dem Gastgeber-Team.

WIR SIND TEAM

WIR REVIEWEN TEAM / PROJEKT

1 · Ausprobieren

WAS HAT SOFORT GEKLAPPT?

WO WUSSTEN WIR NICHT WEITER? GAB ES ABSTÜRZE ODER UNERWARTETE AUSGABEN?

2 · Code lesen Dateiname: _____

LEITFRAGE	UNSERE BEOBACHTUNG
Verstehen wir die Namen von Variablen und Funktionen ohne Erklärung?	
Erledigt jede Funktion eine klar erkennbare Aufgabe?	
Gibt es Code, der mehrfach fast gleich vorkommt?	
Welche Inhalte aus der Checkliste (Teil 4) entdecken wir?	

3 · Zwei Sterne und ein Wunsch

★ DAS IST GUT GELUNGEN

★ DAS IST AUCH GUT GELUNGEN

→ UNSER WUNSCH: DAS WÜRDEN WIR VERBESSERN

VOM GASTGEBER-TEAM AUSZUFÜLLEN

DAS SETZEN WIR ALS NÄCHSTES UM

ANHANG C · VORLAGE

Bogen Präsentation

Ein Bogen pro präsentierendem Team. Die Leitfragen helfen dir beim Beobachten.

MEIN NAME

PRÄSENTIERENDES TEAM / PROJEKT

Leitfragen

BEDIENUNG

Kam die fremde Person ohne Hilfe zurecht? Wo hat sie gezögert? Was passierte bei falschen Eingaben?

CODE

Waren Namen und Funktionen verständlich? Was war besonders geschickt gelöst? Welche Seminarinhalte habe ich entdeckt?

ERKLÄRUNG

Konnte jede Person ihre Funktion Zeile für Zeile erklären? Was habe ich dabei selbst gelernt?

ZUSAMMENARBEIT

Wurde klar, wer was gebaut hat und wie die Teile zusammenpassen? Wie hat das Team Probleme gelöst?

MEINE BEOBACHTUNGEN

DAS HAT MIR BESONDERS GEFALLEN

MEIN TIPP: DAS WÜRD EICH ANDERS MACHEN

MEINE FRAGE AN DAS TEAM

ANHANG D · WERKZEUG

Coding Conventions und Clean Code

Code wird viel öfter gelesen als geschrieben, von deinem Team, vom Ausbilder und von dir selbst in zwei Wochen. Einheitliche Regeln machen ihn leichter lesbar.

Python-Konventionen (PEP 8)

PEP 8 ist der offizielle Stilleitfaden für Python. Die wichtigsten Regeln:

THEMA	REGEL	BEISPIEL
Einrückung	Immer 4 Leerzeichen pro Ebene, keine Tabs mischen.	<code>print(x)</code>
Variablen, Funktionen	Kleinbuchstaben, Wörter mit Unterstrich verbunden (snake_case).	<code>anzahl_versuche</code> <code>def zeige_menue():</code>
Konstanten	Werte, die sich nie ändern: Großbuchstaben, oben in der Datei.	<code>MAX_VERSUCHE = 3</code>
Umlaute	In Namen keine Umlaute und kein ß verwenden. In Texten sind sie erlaubt.	<code>groesse</code> statt <code>größe</code>
Leerzeichen	Um <code>=</code> und Rechenzeichen, nach Kommas. Nicht direkt innerhalb von Klammern.	<code>summe = a + b</code> <code>print(a, b)</code>
Leerzeilen	Zwei Leerzeilen vor jeder Funktion. Innerhalb von Funktionen sparsam, um Abschnitte zu trennen.	
Imports	Ganz oben in der Datei, jeder Import in einer eigenen Zeile.	<code>import random</code>
Zeilenlänge	Höchstens etwa 79 Zeichen. Lange Texte lieber aufteilen.	

PYCHARM HILFT

Code > Reformat Code (Windows: `Strg + Alt + L`, macOS: `⌘ + ⌘ + L`) formatiert deine Datei nach PEP 8. Gelbe Wellenlinien zeigen Stilprobleme. Fahr mit der Maus darüber, dann siehst du den Hinweis.

Docstrings und Kommentare

Ein **Docstring** steht in dreifachen Anführungszeichen direkt unter `def` und sagt in einem Satz, was die Funktion tut. PyCharm zeigt ihn an, wenn jemand die Funktion benutzt. Ein **Kommentar** mit `#` erklärt, warum etwas so gelöst ist.

```
MINDESTLAENGE = 8
```

```
def ist_sicher(password):
    """Gibt True zurück, wenn das Passwort lang genug ist und eine Ziffer enthält."""
    # Länge zuerst prüfen: Das ist die häufigste Schwachstelle
    if len(password) < MINDESTLAENGE:
        return False
    for zeichen in password:
        if zeichen.isdigit():
            return True
    return False
```

ANHANG D · WERKZEUG

Clean-Code-Regeln

Clean Code heißt: Code, den andere ohne Erklärung verstehen und gefahrlos ändern können.

1

Sprechende Namen

Der Name sagt, was drinsteht oder was passiert:
`anzahl_fehler` statt `x`, `berechne_summe()`
 statt `f()`.

2

Eine Funktion, eine Aufgabe

Wenn du die Aufgabe einer Funktion nur mit „und“ beschreiben kannst, teile sie auf.

3

Keine magischen Zahlen

Eine nackte `3` im Code erklärt sich nicht.
`MAX_VERSUCHE = 3` schon, und du änderst sie
 an einer Stelle.

4

Nicht wiederholen

Kommt fast derselbe Code mehrmals vor, gehört er in eine Schleife oder eine Funktion.

5

Kommentare erklären das Warum

Was der Code tut, zeigen gute Namen. Kommentare erklären, warum etwas so gelöst ist. Jede Funktion bekommt einen Docstring.

6

Eingaben prüfen

Rechne damit, dass Nutzende etwas Falsches eingeben. Frag freundlich erneut, statt abzustürzen.

Clean-Code-Durchgang

1. Geh deine eigene Datei mit dem Check unten durch und verbessere sie.
2. Tauscht paarweise: Lies die Datei einer anderen Person. Verstehst du jede Funktion ohne Erklärung? Notiere unklare Stellen und sprech darüber.
3. Startet danach das ganze Programm. Aufräumen darf nichts kaputt machen.

CLEAN-CODE-CHECK FÜR DEINE DATEI

- ☐ Alle Namen verraten, was drinsteht oder was passiert.
- ☐ Keine Funktion ist länger als etwa 20 Zeilen.
- ☐ Feste Werte stehen als Konstanten oben in der Datei.
- ☐ Kein Codeblock kommt mehrfach fast gleich vor.
- ☐ Jede Funktion hat einen Docstring.
- ☐ Falsche Eingaben führen nicht zum Absturz.
- ☐ Kein auskommentierter alter Code, keine ungenutzten Variablen.
- ☐ „Reformat Code“ in PyCharm ausgeführt, keine gelben Wellenlinien mehr.

ANHANG D · WERKZEUG

Clean Code: vorher und nachher

Beide Programme lösen dieselbe Aufgabe. Welches würdest du lieber erweitern?

VORHER

```
import random
z = random.randint(1, 20)
v = 0
while v < 5:
    e = int(input("Zahl: "))
    v = v + 1
    if e == z:
        print("Richtig!")
        v = 5
    elif e < z:
        print("Zu klein")
        print("Noch", 5 - v, "Versuche")
    else:
        print("Zu groß")
        print("Noch", 5 - v, "Versuche")
```

NACHHER

```
import random

MAX_VERSUCHE = 5
OBERGRENZE = 20

def gib_hinweis(tipp, geheimzahl):
    """Gibt einen Hinweis-Text zurück."""
    if tipp < geheimzahl:
        return "Zu klein"
    return "Zu groß"

def spiele_runde():
    """Eine Runde Zahlenraten."""
    geheimzahl = random.randint(1, OBERGRENZE)
    for versuch in range(1, MAX_VERSUCHE + 1):
        tipp = int(input("Zahl: "))
        if tipp == geheimzahl:
            print("Richtig!")
            return
        print(gib_hinweis(tipp, geheimzahl))
        verbleibend = MAX_VERSUCHE - versuch
        print("Noch", verbleibend, "Versuche")
    print("Leider verloren. Die Zahl war", geheimzahl)
```

Was hat sich verbessert?

- **Namen:** `tipp`, `geheimzahl` und `versuch` statt `e`, `z` und `v`.
- **Konstanten** statt magischer Zahlen: Die 5 und die 20 stehen nur noch an einer Stelle.
- **Keine Wiederholung:** Die Zeile „Noch ... Versuche“ steht nur noch einmal.
- **Funktionen** mit Docstring: Jede hat eine klar benannte Aufgabe und lässt sich einzeln testen.
- **Kein Trick mehr:** `v = 5` zum Beenden der Schleife war schwer zu verstehen. `return` sagt klar, dass die Runde vorbei ist.

ANHANG E · WERKZEUG

Basics: Arbeiten mit GitHub

Den Code für den Workshop holt ihr euch aus einem GitHub-Repository. So teilen Teams in der Softwareentwicklung ihren Code.

Die wichtigsten Begriffe

BEGRIFF	BEDEUTUNG
Git	Ein Programm auf deinem Rechner, das Versionen von Dateien verwaltet. Ist bei euch schon installiert.
GitHub	Eine Plattform, auf der Git-Repositorys gespeichert und geteilt werden.
Repository	Ein Projektordner mit allen Dateien und ihrer Versionsgeschichte, kurz „Repo“.
Clone	Eine Kopie eines Repositorys von GitHub auf deinen Rechner holen. Das machst du einmal.
Add	Geänderte Dateien für den nächsten Commit vormerken.
Commit	Den aktuellen Stand der vorgemerkten Dateien als Version speichern, mit einer kurzen Nachricht. Passiert nur auf deinem Rechner.
Push	Deine Commits auf GitHub hochladen, damit andere sie sehen.
Pull	Neue Commits von GitHub in deine Kopie holen.

Das Workshop-Repository holen

1 Einladung annehmen

Das Repository ist privat. Du bekommst eine Einladung per E-Mail an deine GitHub-Adresse. Klicke dort auf **Accept invitation**.

2 Klonen im Terminal

Öffne ein Terminal (Windows: PowerShell), wechsle in den Ordner, in dem das Repository landen soll, und gib ein:

```
cd Dokumente
git clone https://github.com/totoongh/Workshop.git
```

Beim ersten Mal öffnet sich ein Anmeldefenster. Melde dich dort mit deinem GitHub-Konto an. Danach liegt der Ordner `Workshop` in `Dokumente`.

3 In PyCharm öffnen

Menü **File > Open** und den geklonten Ordner auswählen, oder direkt euren Projektordner (Teil 7).

ANHANG E · WERKZEUG

Änderungen speichern und hochladen

Diese Befehle brauchst du in jedem Softwareprojekt, das mit Git arbeitet. Du kannst sie in einem eigenen Repository ausprobieren.

Der Grundablauf: status, add, commit, push

```
git status           # Was hat sich geändert?
git add password.py  # Datei für den Commit vormerken
git commit -m "Passwortlänge abfragen" # Version mit Nachricht speichern
git push             # Commits auf GitHub hochladen
```

Tipps: Commit-Nachrichten kurz und konkret halten, z. B. "Menüpunkt Passwort prüfen ergänzt" statt "Update". Lieber nach jedem funktionierenden Teilschritt einen kleinen Commit als einen großen am Ende.

Spickzettel: wichtige Befehle

BEFEHL	WAS ER TUT
<code>git clone <Adresse></code>	Repository einmalig auf den Rechner holen
<code>git status</code>	Zeigen, welche Dateien geändert oder neu sind
<code>git add <Datei></code> · <code>git add .</code>	Eine Datei oder alle Änderungen für den Commit vormerken
<code>git commit -m "Nachricht"</code>	Vorgemerkte Änderungen als Version speichern
<code>git push</code>	Eigene Commits auf GitHub hochladen
<code>git pull</code>	Neue Commits von GitHub holen
<code>git log --oneline</code>	Die bisherigen Commits kurz auflisten
<code>cd <Ordner></code> · <code>cd ..</code>	In einen Ordner wechseln · eine Ebene nach oben
<code>dir</code> (Windows) · <code>ls</code> (macOS)	Inhalt des aktuellen Ordners anzeigen
<code>py datei.py</code> · <code>python3 datei.py</code>	Ein Python-Programm im Terminal starten

Tipp: Im Terminal ergänzt die **Tab-Taste** Ordner- und Dateinamen. Mit **Pfeil nach oben** holst du den letzten Befehl zurück.

Wenn etwas nicht klappt

MELDUNG	LÖSUNG
Repository not found	Einladung noch nicht angenommen, oder du bist mit einem anderen GitHub-Konto angemeldet.
Authentication failed	Anmeldung abgelaufen oder abgebrochen. Befehl noch einmal ausführen und neu anmelden.
Please tell me who you are	Git kennt deinen Namen noch nicht. Einmalig eingeben: <code>git config --global user.name "Vorname Nachname"</code> <code>git config --global user.email "deine@adresse.de"</code>
rejected ... fetch first	Auf GitHub gibt es neuere Commits. Erst <code>git pull</code> , dann erneut <code>git push</code> .

ANHANG F · WERKZEUG

Bibliotheken: Standardmodule und pip

Niemand programmiert alles selbst. Für viele Aufgaben gibt es fertigen Code, den du in dein Programm einbindest.

Zwei Arten von Bibliotheken

STANDARDMODULE

Sind in Python schon enthalten. Du bindest sie mit `import` ein und kannst sie sofort nutzen.

Beispiele: `random`, `math`, `time`, `datetime`, `string`

Für alle im Workshop. Mindestens ein Modul gehört in euer Programm.

EXTERNE PAKETE (PIP)

Musst du vorher installieren. Sie kommen aus dem Python Package Index (PyPI), einer Sammlung von über 500.000 Paketen.

Beispiele: `rich`, `pyfiglet`, `pygame`

Exkurs für Fortgeschrittene, siehe nächste Seite.

So bindest du ein Modul ein

```
import random                # ganzes Modul: random.randint(1, 6)
from random import randint    # nur eine Funktion: randint(1, 6)
import datetime as dt         # mit Kurznamen: dt.date.today()
```

Am verständlichsten ist meist die erste Form. Man sieht beim Lesen sofort, woher eine Funktion kommt.

Nützliche Standardmodule

MODUL	BEISPIEL	WOFÜR
<code>random</code>	<code>random.randint(1, 6)</code> <code>random.choice(liste)</code> <code>random.shuffle(liste)</code>	Zufallszahl, zufälliges Element, Liste mischen
<code>math</code>	<code>math.sqrt(16)</code> · <code>math.ceil(4.2)</code>	Wurzel, Aufrunden, <code>math.pi</code>
<code>time</code>	<code>time.sleep(1)</code>	Eine Sekunde warten, z. B. für Spannung im Spiel
<code>datetime</code>	<code>date.today()</code> nach <code>from datetime import date</code>	Heutiges Datum, mit Daten rechnen
<code>string</code>	<code>string.ascii_letters</code> <code>string.digits</code>	Fertige Zeichenvorräte, z. B. für Passwörter

SELBST NACHSCHLAGEN

`help(random.randint)` in der Python-Konsole zeigt die Beschreibung einer Funktion. Alle Standardmodule sind unter docs.python.org/3/library beschrieben. In PyCharm zeigt `Strg + Q` (macOS: `F1`) die Doku zu der Funktion unter dem Cursor.

Beispielprogramm im Repository: `beispiele/01_standardmodule.py`

ANHANG F · EXKURS FÜR FORTGESCHRITTENE

Externe Pakete mit pip

pip ist das Installationsprogramm für Python-Pakete. Es lädt ein Paket aus dem Python Package Index und installiert es für deinen Python-Interpreter.

Ein Paket installieren

IN PYCHARM

Unten links das Fenster **Python Packages** öffnen, den Paketnamen suchen, z. B. `rich`, und auf **Install** klicken.

Alternativ: **Settings > Project > Python Interpreter**, dann auf **+** klicken.

IM TERMINAL (AUCH IN PYCHARM UNTEN)

```
py -m pip install rich
```

macOS: `python3 -m pip install rich`

Prüfen, was installiert ist: `py -m pip list`

Ein Paket nutzen

```
from rich.console import Console

konsole = Console()
konsole.print("Hallo [bold green]Workshop[/bold green]!")
```

Beispielprogramm mit Tabelle im Repository: `beispiele/02_pip_rich.py`

Pakete für eure Projekte

PAKET	WOFÜR
<code>rich</code>	Farbige Ausgaben, Tabellen und Fortschrittsbalken in der Konsole
<code>pyfiglet</code>	Große Überschriften aus Textzeichen, z. B. für einen Spieletitel
<code>colorama</code>	Einfache Textfarben in der Konsole, auch unter Windows
<code>pygame</code>	Spiele mit Grafik und Tastatursteuerung. Nur für sehr Fortgeschrittene.

DARAUF MUSST DU ACHTEN

- **Alle im Team brauchen das Paket.** Notiert es in einer Datei `requirements.txt` im Projektordner, ein Paket pro Zeile. Dann installiert jede Person alles mit `py -m pip install -r requirements.txt`.
- **Nur bekannte Pakete installieren.** Achte auf die genaue Schreibweise. Es gibt gefälschte Pakete mit ähnlichen Namen.
- **Klappt die Installation nicht**, zum Beispiel wegen des Firmennetzes, frag den Ausbilder. Bastle nicht an den Netzwerkeinstellungen.
- PyCharm legt für jedes Projekt eine eigene Umgebung an (`.venv`). Ein Paket, das du in einem Projekt installierst, fehlt deshalb im nächsten Projekt.

ANHANG G · WERKZEUG

Fehlerbehandlung mit try und except

Gibt jemand bei `int(input())` Buchstaben ein, stürzt dein Programm ab. Mit `try` und `except` fängst du solche Fehler ab und reagierst freundlich. In anderen Sprachen wie Java heißt das `try` und `catch`.

Die Idee: Ausnahmen

Kann Python eine Zeile nicht ausführen, löst es eine **Ausnahme** (englisch *Exception*) aus. Ohne Behandlung bricht das Programm ab und zeigt eine Fehlermeldung. Die letzte Zeile nennt die Art des Fehlers, hier `ValueError`:

```
Zahl: abc
Traceback (most recent call last):
  File "main.py", line 1, in <module>
    zahl = int(input("Zahl: "))
ValueError: invalid literal for int() with base 10: 'abc'
```

Der Aufbau

```
try:
    zahl = int(input("Zahl: "))          # das hier könnte schiefgehen
    print("Doppelt:", zahl * 2)
except ValueError:
    print("Das war keine ganze Zahl.")  # das passiert stattdessen
    print("Weiter geht's")
```

1. Python versucht, den `try`-Block auszuführen.
2. Klappt alles, wird der `except`-Block übersprungen.
3. Tritt ein `ValueError` auf, springt Python sofort in den `except`-Block. Die restlichen Zeilen im `try`-Block werden übersprungen.
4. Danach läuft das Programm ganz normal weiter. Es stürzt nicht ab.

Das wichtigste Muster: fragen, bis die Eingabe passt

```
def frage_zahl(text):
    """Fragt so lange, bis eine ganze Zahl eingegeben wird."""
    while True:
        try:
            return int(input(text))
        except ValueError:
            print("Bitte eine ganze Zahl eingeben.")

alter = frage_zahl("Dein Alter: ")
```

`while True` wiederholt endlos. Erst wenn `int()` klappt, beendet `return` die Funktion und damit auch die Schleife. Diese Funktion kann euer ganzes Team nutzen.

Lauffähig im Repository: `beispiele/08_fehlerbehandlung.py`

ANHANG G · NACHSCHLAGEN

Fehlerbehandlung: zum Nachschlagen

Die häufigsten Fehlerarten und wie du mehrere davon abfängst.

Häufige Fehlerarten

FEHLERART	AUSLÖSER ZUM BEISPIEL	BEDEUTUNG
<code>ValueError</code>	<code>int("abc")</code>	Der Wert passt nicht, obwohl der Typ stimmt
<code>ZeroDivisionError</code>	<code>10 / 0</code>	Division durch null
<code>IndexError</code>	<code>liste[5]</code> bei drei Elementen	Diese Position gibt es in der Liste nicht
<code>KeyError</code>	<code>geraet["farbe"]</code>	Diesen Schlüssel gibt es im Dictionary nicht
<code>TypeError</code>	<code>"5" + 5</code>	Die Typen passen nicht zusammen
<code>FileNotFoundError</code>	<code>open("fehlt.txt")</code>	Die Datei gibt es nicht (Anhang H)

Mehrere Fehlerarten abfangen

```
try:
    anzahl = int(input("Anzahl Personen: "))
    pro_person = 100 / anzahl
    print("Pro Person:", pro_person)
except ValueError:
    print("Bitte eine Zahl eingeben.")
except ZeroDivisionError:
    print("Die Anzahl darf nicht 0 sein.")
```

else und finally

Ein `else`-Block läuft nur, wenn im `try`-Block **kein** Fehler aufgetreten ist. Ein `finally`-Block läuft **immer**, mit oder ohne Fehler, zum Beispiel für eine Abschlussmeldung.

Typische Stolpersteine

PROBLEM	BESSER SO
<code>except:</code> ohne Fehlerart	Fängt alles ab, auch Tippfehler in deinem eigenen Code. Gib immer die konkrete Fehlerart an.
Viel Code im <code>try</code> -Block	Nur die Zeilen hineinschreiben, die wirklich schiefgehen können. Sonst weißt du nicht, welche Zeile den Fehler ausgelöst hat.
Fehler still verschlucken	<code>except ValueError: pass</code> sagt niemandem, was los ist. Gib immer eine verständliche Meldung aus.
<code>try</code> statt <code>if</code>	Was du einfach prüfen kannst, etwa eine Menüauswahl, prüfst du mit <code>if</code> . <code>try</code> ist für Fälle, die du nicht vorher prüfen kannst.

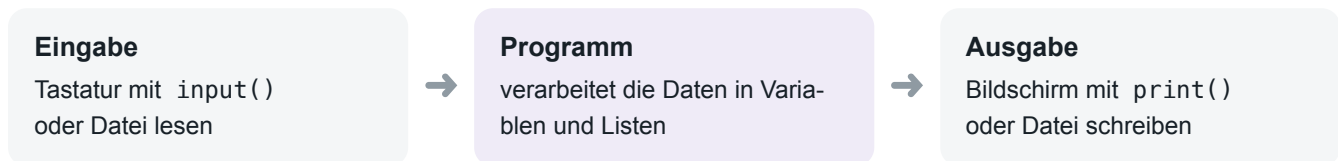
ANHANG H · EXKURS FÜR FORTGESCHRITTENE

Dateien und Datenströme

Bisher vergisst euer Programm alles, sobald es endet. Mit Dateien bleiben Daten erhalten: Highscores, Inventar, Tickets.

Die Idee: Daten fließen

Ein Programm bekommt Daten von irgendwoher und gibt Daten irgendwohin ab. Man spricht von **Datenströmen**, weil die Daten Zeichen für Zeichen hinein- und hinausfließen. Bisher kennst du zwei Ströme. Dateien sind zwei weitere.



Der Unterschied: Was auf dem Bildschirm steht, ist nach dem Programmende weg. Was in einer Datei auf der Festplatte steht, bleibt.

Eine Datei öffnen

```
with open("notizen.txt", "r", encoding="utf-8") as datei:
    inhalt = datei.read()
    print(inhalt)
```

- `open()` öffnet die Datei und gibt ein Dateiojekt zurück. Hier heißt es `datei`.
- `with ... as ...:` sorgt dafür, dass die Datei am Ende des eingerückten Blocks automatisch geschlossen wird, auch wenn ein Fehler passiert.
- `encoding="utf-8"` sorgt dafür, dass Umlaute auf jedem Rechner richtig ankommen.

MODUS	BEDEUTUNG	ACHTUNG
"r"	lesen (read)	Fehler, wenn die Datei nicht existiert
"w"	schreiben (write)	Legt die Datei neu an. Vorhandener Inhalt wird gelöscht!
"a"	anhängen (append)	Schreibt ans Ende, legt die Datei bei Bedarf an

ANHANG H · BASICS

Dateien: die Grundlagen zum Nachschlagen

Die wichtigsten Bausteine. Alle Beispiele findest du lauffähig im Ordner `beispiele`.

Schreiben, anhängen und lesen

```
import os

DATEI = "liste.txt"

with open(DATEI, "w", encoding="utf-8") as datei:    # neu anlegen
    datei.write("Laptop\n")
    datei.write("Headset\n")

with open(DATEI, "a", encoding="utf-8") as datei:    # ans Ende anhängen
    datei.write("Maus\n")

geraete = []
if os.path.exists(DATEI):                            # gibt es die Datei?
    with open(DATEI, "r", encoding="utf-8") as datei:
        for zeile in datei:
            geraete.append(zeile.strip())
print(geraete)                                       # ['Laptop', 'Headset', 'Maus']
```

- `write()` macht keinen Zeilenumbruch. Den schreibst du selbst mit `\n`. Zahlen wandelst du vorher mit `str()` um.
- `strip()` entfernt beim Lesen den Zeilenumbruch am Ende jeder Zeile.
- `os.path.exists()` prüft vorher, ob es die Datei gibt. Sonst bricht `"r"` mit einem Fehler ab.

Mehrere Werte pro Zeile: Trennzeichen

Speichere zusammengehörige Werte in einer Zeile, getrennt durch ein Semikolon. Beim Lesen zerlegt `split(";")` die Zeile wieder in eine Liste.

```
def speichere_highscore(name, punkte):
    with open("highscores.txt", "a", encoding="utf-8") as datei:
        datei.write(name + ";" + str(punkte) + "\n")

# Zeile in der Datei: Ada;42
teile = "Ada;42".split(";")    # ['Ada', '42']
punkte = int(teile[1])         # 42
```

Ganzes Beispiel mit Laden und Anzeigen: `beispiele/04_highscore.py`. Für Listen mit Dictionaries gibt es das Modul `json`: `beispiele/05_json_speichern.py`.

Typische Stolpersteine

PROBLEM	URSACHE UND LÖSUNG
<code>FileNotFoundError</code>	Die Datei gibt es (noch) nicht, oder sie liegt in einem anderen Ordner. Vorher mit <code>os.path.exists()</code> prüfen oder mit <code>try</code> und <code>except</code> abfangen (Anhang G).
Datei ist plötzlich leer	Mit <code>"w"</code> statt <code>"a"</code> geöffnet. <code>"w"</code> löscht den alten Inhalt.
Alles steht in einer Zeile	<code>\n</code> am Ende von <code>write()</code> vergessen.
Umlaute sehen kaputt aus	<code>encoding="utf-8"</code> fehlt beim Schreiben oder Lesen.

ANHANG I · EXKURS FÜR FORTGESCHRITTENE

Grafische Oberflächen mit tkinter

Statt Text im Terminal: ein Fenster mit Eingabefeldern und Knöpfen. tkinter ist in Python enthalten und reicht für kleine Werkzeuge und Spiele.

Die Idee: Das Programm wartet auf Ereignisse

KONSOLEPROGRAMM

Das Programm bestimmt den Ablauf. Es läuft von oben nach unten und wartet bei `input()`, bis du etwas eingibst.

PROGRAMM MIT OBERFLÄCHE

Die Nutzerin oder der Nutzer bestimmt den Ablauf. Das Programm baut das Fenster auf und wartet dann in `mainloop()` auf **Ereignisse**, zum Beispiel einen Klick. Bei einem Klick ruft es die Funktion auf, die zu diesem Knopf gehört.

Ein erstes Fenster

```
import tkinter as tk

def begrüessen():
    name = eingabe.get()
    ausgabe.config(text="Hallo " + name + "!")

fenster = tk.Tk()
fenster.title("Mein erstes Fenster")

tk.Label(fenster, text="Wie heißt du?").pack(padx=20, pady=(20, 5))
eingabe = tk.Entry(fenster)
eingabe.pack(padx=20)
tk.Button(fenster, text="Begrüßen", command=begrüessen).pack(pady=10)
ausgabe = tk.Label(fenster, text="")
ausgabe.pack(padx=20, pady=(0, 20))

fenster.mainloop()
```

1. `tk.Tk()` erzeugt das Fenster.
2. Jedes Element (Widget) bekommt das Fenster als ersten Parameter. `pack()` setzt es untereinander ins Fenster.
3. `command=begrüessen` verbindet den Knopf mit der Funktion. **Ohne Klammern!** Mit Klammern würde die Funktion sofort einmal aufgerufen.
4. `fenster.mainloop()` zeigt das Fenster und wartet auf Ereignisse. Code danach läuft erst, wenn das Fenster geschlossen ist.

Lauffähig im Repository: `beispiele/06_gui_minimal.py`

ANHANG I · BASICS

tkinter: die Grundlagen zum Nachschlagen

Die wichtigsten Widgets und ein Tipp, der dir viel Arbeit spart.

Wichtige Widgets

WIDGET	ERZEUGEN	WOFÜR
Label	<code>tk.Label(fenster, text="...")</code>	Text anzeigen. Ändern mit <code>.config(text="...")</code>
Entry	<code>tk.Entry(fenster, width=10)</code>	Einzeiliges Eingabefeld. Lesen mit <code>.get()</code> , liefert immer einen <code>str</code>
Button	<code>tk.Button(fenster, text="OK", command=f)</code>	Knopf, ruft beim Klick die Funktion <code>f</code> auf
Listbox	<code>tk.Listbox(fenster)</code>	Liste anzeigen. Einfügen mit <code>.insert(tk.END, "...")</code>
messagebox	<code>messagebox.showinfo("Titel", "Text")</code>	Hinweisfenster. Vorher: <code>from tkinter import messagebox</code>

Anordnen mit grid()

Statt `pack()` kannst du Widgets mit `grid(row=0, column=1)` in Zeilen und Spalten anordnen, zum Beispiel Beschriftung links und Eingabefeld rechts. Verwende in einem Fenster entweder `pack()` oder `grid()`, nicht beides. Ein vollständiges Beispiel mit `grid()` steht in `beispiele/07_gui_passwort.py`.

DER WICHTIGSTE TIPP: LOGIK UND OBERFLÄCHE TRENNEN

Programmiere dein Feature zuerst als Konsolenversion mit einer Funktion, die etwas berechnet und mit `return` zurückgibt. Die Oberfläche ruft dann nur noch diese Funktion auf. So kannst du die Logik ohne Fenster testen und deine bisherige Arbeit weiterverwenden.

```
def erzeuge_passwort(laenge):           # Logik: kennt kein Fenster
    ...
    return password

def knopf_geklickt():                   # Oberfläche: liest, ruft auf, zeigt an
    laenge = int(eingabe_laenge.get())
    ausgabe.config(text=erzeuge_passwort(laenge))
```

Vollständiger Passwort-Generator mit Prüfung der Eingabe: `beispiele/07_gui_passwort.py`

Typische Stolpersteine

PROBLEM	URSACHE UND LÖSUNG
Funktion läuft sofort beim Start, Klick tut nichts	<code>command=f()</code> statt <code>command=f</code>
Fenster erscheint nicht	<code>fenster.mainloop()</code> fehlt am Ende
Rechnen mit der Eingabe klappt nicht	<code>.get()</code> liefert Text. Mit <code>int()</code> umwandeln